

EVENTPULSE

Enterprise Event Platform — Functional & Technical Specification

Version 2.0 • AWS Production Architecture • Complete User View & User Flow

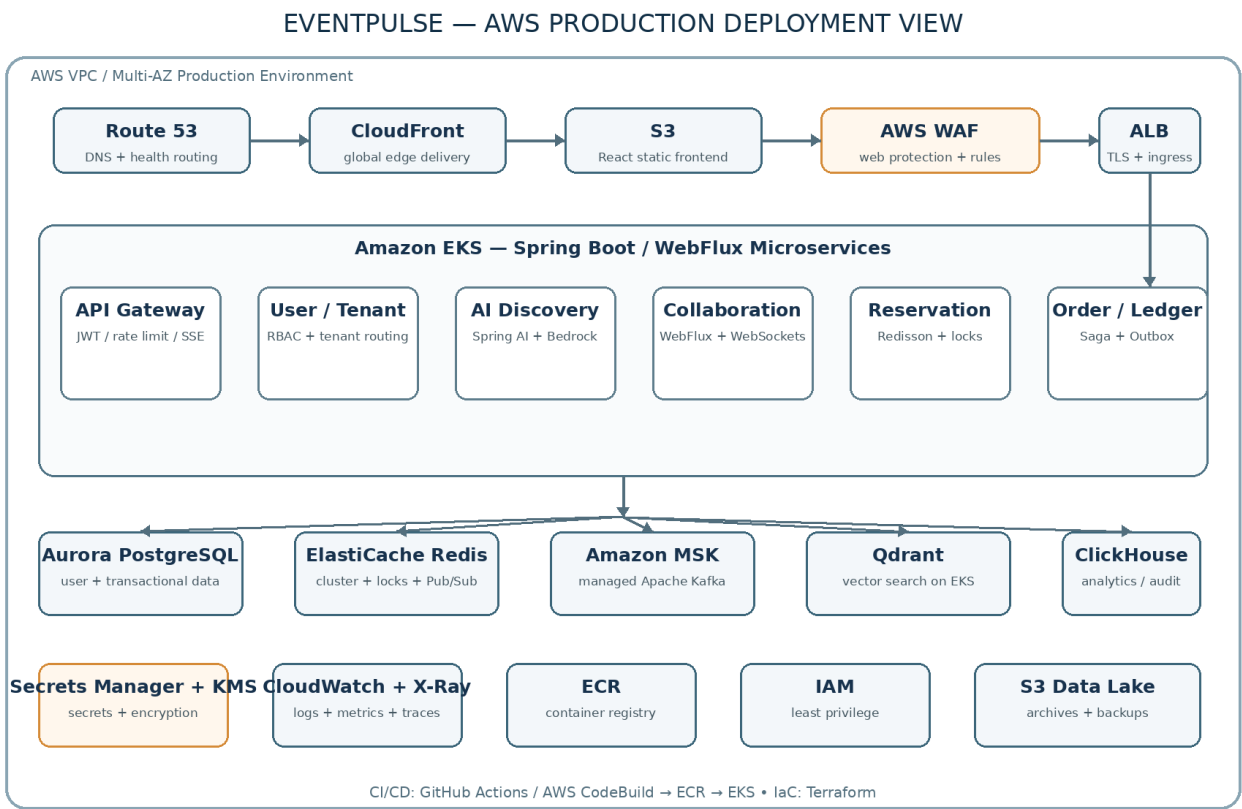


Figure 1 — AWS production deployment view.

Purpose: define what the user sees, how the user moves through EventPulse, what each screen does, which service backs it, what data is created, and how the system behaves under concurrency and failure.

Architecture: React 18 + TypeScript • Spring Boot 3 • Java 21 • Spring WebFlux • Spring Cloud Gateway • Redis Cluster • Apache Kafka / Amazon MSK • PostgreSQL • Qdrant • ClickHouse • Amazon EKS.

1. Document Purpose & Product Definition

EventPulse is a multi-tenant event discovery, collaborative seat-planning, reservation and ticketing platform. The product is designed for high-concurrency event sales while giving users a modern AI-assisted discovery experience and real-time group booking.

Version 2.0 defines the product from two perspectives simultaneously: the user experience and the distributed backend. Every major user action is mapped to a UI state, API/protocol, service owner, persistence layer and failure behavior.

The platform is designed around four core product outcomes:

- Discover: users can search for events using natural language rather than rigid filters.
- Collaborate: friends can plan and inspect the same seating map in real time.
- Reserve: seats are temporarily held so users have time to complete checkout.
- Purchase: checkout is idempotent and financially consistent even when distributed steps fail.

The architecture targets flash-sale conditions where many clients can attempt the same inventory resource at nearly the same time. The design therefore treats inventory locking, idempotency and workflow recovery as first-class requirements rather than implementation details.

2. User View & Product Experience

2.1 Primary user personas

Persona	What they need	EventPulse experience
Individual buyer	Find an event quickly and purchase seats without unnecessary interaction.	AI discovery → event details → seating map → temporary hold → checkout → digital ticket.
Group planner	Coordinate seat selection with friends in real time.	Create room → invite friends → shared seating map → live cursors/hover state → reserve seats.
Tenant administrator	Manage users and access within an organization/tenant.	RBAC-controlled administration and tenant-scoped data.
Platform operator	Monitor reliability, throughput and business health.	Operational dashboards, Kafka metrics, audit events and alerting.

2.2 User-facing application structure

- Top navigation: Discover, My Events/Tickets, Reservations, Profile and tenant-aware account controls.
- Discovery workspace: conversational search, filters, event cards and recommendations.
- Event detail: event information, venue/date, price range, availability and Select Seats action.
- Seating workspace: Canvas-based interactive seat map with legend, selected seats, availability and booking mode.
- Group room: room code, participant list, presence, live cursor indicators and shared seat state.
- Checkout: order summary, timer, payment form/status and final confirmation.
- Ticket view: digital ticket, event details, order reference and ticket state.
- System feedback: loading, success, warning, conflict, session expiry and retry states are explicit UI states rather than silent failures.

2.3 UX principles

- The user should always know whether a seat is available, held by them, unavailable, or being changed by another participant.
- The 10-minute reservation timer is visible throughout checkout.
- A failed request should explain whether the issue is a conflict, expiry, payment failure or temporary system problem.
- Group collaboration should feel live, but the authoritative booking decision remains server-side.
- AI output is streamed progressively so the interface does not appear blocked while recommendations are generated.

3. Detailed User Screen Specification

Screen	What user sees	Primary action	Protocol	Backend owner	State
Screen 1 — AI Discovery Dashboard	Search input, suggested prompts, filters, event cards, streamed AI response.	Natural-language query; optional filters such as price, location and date.	SSE + REST	AI Discovery Service	Qdrant + event metadata
Screen 2 — Event Details	Event title, venue, date/time, description, pricing and Select Seats CTA.	Select event; review details; proceed to seating.	REST	Gateway → Event/Discovery domain	PostgreSQL / indexed event data
Screen 3 — Interactive Seating Map	Canvas seat map, legend, selected-seat panel, booking mode selector.	Choose Solo Booking or Group Plan; select seats.	REST or WebSocket/STOMP	Reservation / Collaboration	Redis + PostgreSQL
Screen 4 — Group Planning Room	Room code, participant presence, live cursors, seat hover/selection state.	Invite/join friends; coordinate seats.	STOMP/WebSocket + Redis Pub/Sub	Collaboration Service	Redis transient state
Screen 5 — Reservation / Checkout	Countdown timer, selected seats, price breakdown, payment form.	Confirm order and pay; retry safely if network request repeats.	REST	Order & Ledger	Redis idempotency + PostgreSQL
Screen 6 — Payment Processing	Progress state; success/failure/retry message.	Wait for workflow result; do not submit duplicate payment.	REST + WebSocket	Order & Ledger	Saga state + ledger DB
Screen 7 — Confirmation / Ticket	Ticket, order reference, event information, status.	View/save ticket; return to My Events.	WebSocket/REST	Order + Ticket workflow	PostgreSQL + event projections
Screen 8 — My Events	Upcoming tickets, past events, reservation/order history.	Open ticket or inspect order status.	REST	User/Order domain	PostgreSQL / projections

The screen model deliberately separates user-visible state from backend authority. For example, the browser can optimistically highlight a seat, but only the Reservation Service can establish a valid temporary hold.

4. Primary User Flow — Discover to Ticket

- 01 — Open EventPulse — React application loads from CloudFront/S3; authentication context is established.
- 02 — Enter query — User enters “Rock concerts under \$100 near NYC.”
- 03 — AI discovery — Gateway validates JWT/rate limit; AI service performs semantic retrieval and streams response.
- 04 — Select event — User opens event details and chooses Select Seats.
- 05 — Choose mode — User selects Solo Booking or Group Plan.
- 06A — Solo path — Client uses direct REST reservation; no collaboration room is required.
- 06B — Group path — System creates room_id; participants join through STOMP/WebSockets.
- 07 — Select seats — Seat map updates locally; server validates actual availability.
- 08 — Create hold — Reservation Service obtains distributed lock and establishes 10-minute hold.
- 09 — Checkout — User confirms order; X-Idempotency-Key prevents duplicate execution.
- 10 — Saga workflow — Order/Ledger coordinates payment, ledger and ticket steps.
- 11A — Success — Order completes; confirmation is pushed; ticket is rendered.
- 11B — Failure — Saga executes compensation and UI shows the appropriate failure state.
- 12 — Analytics — Kafka events are consumed asynchronously and projected into ClickHouse.

Critical UX invariant: the user may see temporary visual states, but a successful purchase is only shown after the backend has reached a confirmed order state.

5. Group Booking User Flow

Group booking is a separate interaction path because multiple browsers must observe shared state without turning every UI event into a database transaction.

5.1 Group flow

1. Create room — User selects Group Plan. Collaboration Service generates a unique room_id/code.
2. Share room — UI presents a short room code/share action.
3. Join — Friends authenticate and connect to the room using STOMP over WebSockets.
4. Presence — Participants appear in the room; connection state is maintained by the collaboration layer.
5. Explore — Each participant can hover seats and move their cursor. These are transient collaboration events.
6. Coordinate — Seat selections are synchronized through Redis Pub/Sub across collaboration service nodes.
7. Reserve — A final seat choice is sent to Reservation Service. Distributed locking decides ownership.
8. Confirm — Successful hold state is broadcast back to room participants.
9. Checkout — The user proceeds to checkout while the hold timer remains active.
10. Disconnect/reconnect — If a WebSocket connection drops, the client reconnects and requests authoritative room/seat state.

5.2 Collaboration state categories

State	Example	Durability	Transport
Transient UI state	Cursor position, seat hover	Not durable	Redis Pub/Sub + WebSocket
Room membership	Connected participant	Recoverable	Redis/service memory
Seat reservation	Seat held for 10 minutes	Durable + TTL coordination	PostgreSQL + Redis
Purchase	Order/payment/ledger	Durable	PostgreSQL + Kafka

6. Reservation & Seat State Specification

6.1 Seat lifecycle

State	Meaning	Allowed transition
AVAILABLE	Seat can be selected.	AVAILABLE → HELD
HELD	Seat is temporarily reserved by a user/order.	HELD → CONFIRMED or AVAILABLE
CONFIRMED	Seat belongs to a completed purchase.	Terminal for normal booking lifecycle
EXPIRED	Hold elapsed without completion.	EXPIRED → AVAILABLE
CONFLICT	Requested hold lost because another user won the race.	UI refreshes from authoritative state

6.2 Reservation operation

- Client sends event_id, seat_id(s), tenant/user context and request correlation ID.
- Reservation Service creates canonical lock keys such as lock:seat:{event_id}:{seat_id}.
- Redisson obtains the distributed lock.
- Service checks authoritative seat state.
- If AVAILABLE, service writes the hold and creates the Redis TTL state.
- Lock is released after the critical section.
- Client receives the hold expiration timestamp and updates its countdown.
- At expiration, the cleanup path transitions the seat back to AVAILABLE if it was not confirmed.

6.3 Concurrency rule

For N concurrent requests targeting the same seat, at most one request may successfully establish the hold under the same canonical resource key. Competing requests receive a conflict response such as HTTP 409 and must refresh authoritative availability.

6.4 Timer rule

The browser timer is informational. The server-side Redis TTL and durable reservation state determine whether the hold is actually valid. Client clock manipulation must never extend a reservation.

7. Checkout, Payment & Order Specification

7.1 Checkout request

Input	Requirement
Order/cart ID	Identifies the reservation being purchased.
Selected seats	Must match the server-side hold.
Tenant/user identity	Derived from authenticated context.
X-Idempotency-Key	Required UUID-like unique key for the checkout attempt.
Payment data/token	Should use tokenized/approved payment integration rather than storing raw sensitive payment data.
Correlation/workflow ID	Used for distributed tracing and Saga recovery.

7.2 Checkout state machine

State	Meaning
PENDING	Checkout request accepted; workflow started.
PAYMENT_PROCESSING	Payment step executing.
LEDGER_POSTED	Financial entries committed.
TICKET_GENERATED	Ticket creation succeeded.
COMPLETED	All required business steps completed.
FAILED	Workflow failed and compensation completed or is pending.
COMPENSATING	Saga is executing rollback/compensation actions.

7.3 Duplicate request behavior

If the same X-Idempotency-Key is received again, the service must not initiate another payment. It should return the stored result of the original operation or the current workflow status.

8. AI Discovery Functional Specification

8.1 User experience

- Search accepts natural language and does not require users to know the internal event taxonomy.
- The UI displays a streaming response rather than waiting for a single large payload.
- Results should expose enough event metadata for the user to understand why an event was returned.
- Filters can constrain semantic results by structured attributes such as price, location and date.

8.2 Backend flow

React → Gateway → AI Discovery Service → embedding/retrieval → Qdrant → model inference via Spring AI/Bedrock → SSE → React.

8.3 Tenant/security boundary

Retrieval must apply tenant and visibility constraints before results are presented. AI-generated text must never become the source of truth for ticket inventory, price or payment state; those values must come from authoritative backend services.

9. Technical Architecture & AWS Deployment

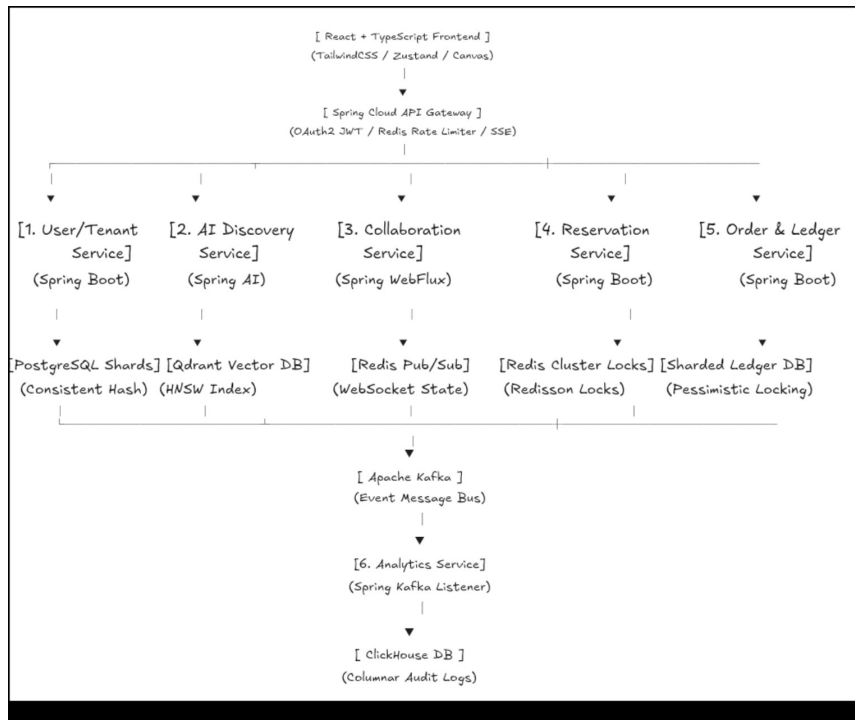


Figure 2 — Original logical service/data architecture.

9.1 AWS production topology

- Route 53 provides DNS.
- CloudFront serves the React frontend from S3.
- AWS WAF protects public HTTP traffic.
- Application Load Balancer provides application ingress.
- Amazon EKS runs Spring Boot/WebFlux services across multiple Availability Zones.
- Amazon ECR stores versioned service images.
- Amazon MSK provides managed Apache Kafka.
- Amazon ElastiCache for Redis provides distributed state and coordination.
- Aurora/RDS PostgreSQL provides transactional persistence.
- Qdrant runs as the vector store for semantic discovery.
- ClickHouse stores analytical and audit projections.
- Secrets Manager, KMS and IAM provide security primitives.
- CloudWatch and distributed tracing provide operational visibility.

10. Service Responsibilities & Integration Contracts

Service	Responsibility	Integration	Primary store
Gateway	Authenticate, authorize, rate-limit, route	REST/SSE/WebSocket handshake	Redis
User/Tenant	Profiles, RBAC, tenant routing	REST + identity context	PostgreSQL
AI Discovery	Semantic search/recommendation	REST + SSE	Qdrant
Collaboration	Rooms, presence, live seat-map events	STOMP/WebSocket	Redis Pub/Sub
Reservation	Seat availability, temporary holds	REST + domain events	Redis + PostgreSQL
Order/Ledger	Orders, payment workflow, accounting	REST + Kafka	PostgreSQL
Analytics/Audit	Consume and project events	Kafka consumer	ClickHouse

10.1 Service ownership rule

No service should directly modify another service's database. Cross-service state changes occur through APIs or versioned domain events. This preserves ownership and allows services to evolve independently.

11. Event Model & Kafka Specification

Event	Producer	Consumers / purpose
SeatHeld	Reservation	Order, Analytics, Collaboration
SeatHoldExpired	Reservation	Inventory recovery, Analytics
OrderCreated	Order	Analytics, workflow consumers
PaymentAuthorized	Order/Ledger	Ledger, Audit
PaymentFailed	Order/Ledger	Saga compensation, Audit
LedgerCompensated	Ledger	Audit, Analytics
OrderCompleted	Order	Ticket/Notification, Analytics
RoomCreated	Collaboration	Audit, Analytics

11.1 Event envelope

event_id, event_type, schema_version, aggregate_id, tenant_id, correlation_id, occurred_at, and payload should be treated as the minimum practical metadata for enterprise event processing.

Consumers must be idempotent. Retry and dead-letter handling should be defined per topic/consumer rather than relying on infinite retries.

12. Failure Scenarios & User Experience

Failure	Backend response	User-facing behavior
Seat already taken	HTTP 409 / authoritative refresh	"That seat was just taken. Please choose another seat."
Reservation expired	Hold no longer valid	"Your seat hold expired. Please select the seats again."
Duplicate checkout	Cached/idempotent result	Do not show a second payment; show original/current order status.
Payment failed	Saga enters compensation	"Payment could not be completed. Your seat hold is being released."
Ticket generation failed	Compensation workflow	Show order recovery status; never claim purchase success prematurely.
WebSocket disconnected	Reconnect + resync	Show "Reconnecting..." and restore room state after reconnection.
AI service slow	Streaming/loading state	Show partial response and allow retry/cancel.
Kafka consumer delayed	Async backlog	User-facing transaction remains decoupled; operators receive lag alerts.

The key UX principle is truthful state: the UI must not represent a seat, payment or ticket as confirmed until the authoritative backend state says it is confirmed.

13. Security & Multi-Tenant Specification

- Authentication: OAuth2/JWT at the gateway; sensitive domain services also enforce authorization.
- Authorization: RBAC and tenant-scoped permissions.
- Tenant identity: derive from trusted authentication claims, not a browser-provided tenant_id.
- Network: databases, Redis, MSK and internal services remain on private network paths.
- Secrets: AWS Secrets Manager; no credentials in source code/images.
- Encryption: TLS in transit and KMS-backed encryption at rest where supported.
- Audit: important domain events are retained in the analytics/audit pipeline.
- AI data boundary: only authorized event/user context reaches the AI retrieval and generation pipeline.

14. Non-Functional Requirements & Acceptance Targets

Requirement	Specification
Availability	Multi-AZ deployment; health checks; graceful degradation; recovery procedures.
Performance	Architectural target of 20,000+ RPS during flash-sale scenarios; validate with load testing.
Real-time sync	Target sub-50ms for collaboration updates under defined test conditions.
Reservation correctness	No successful double allocation for the same seat under concurrency test.
Idempotency	Repeated checkout with the same key produces one logical payment/order.
Recovery	Saga compensates defined partial failures and converges to a valid business state.
Observability	Every critical workflow traceable by correlation/workflow ID.
Scalability	Services scale independently; Kafka and Redis absorb distributed workload patterns.
Security	Tenant isolation and least-privilege access enforced.

Performance disclaimer: 20,000+ RPS and sub-50ms are engineering targets in this specification. They should become project claims only after reproducible benchmark evidence is collected.

15. Validation & Test Strategy

Test class	What must be proven
Unit tests	Domain rules, state transitions, idempotency decisions, compensation logic.
Integration tests	PostgreSQL, Redis, Kafka, Qdrant and service boundaries.
Concurrency tests	Thousands of simultaneous attempts on the same seat; verify one winner.
Load tests	Flash-sale traffic; measure p50/p95/p99 latency, RPS and error rate.
WebSocket tests	Concurrent rooms, reconnects, cross-node Pub/Sub propagation.
Saga failure tests	Inject payment/ticket/ledger failures and verify compensation.
Chaos tests	Kill pods, disconnect Redis/Kafka consumers, restart services and verify recovery.
Security tests	JWT tampering, tenant-boundary tests, authorization failures, secret exposure checks.
End-to-end tests	Discover → seats → hold → checkout → ticket across realistic AWS-like environment.

16. Implementation Roadmap

Phase	Deliverable
Phase 1 — Foundation	React shell, Gateway, User/Tenant, PostgreSQL, authentication.
Phase 2 — Discovery	Event ingestion, Qdrant, Spring AI, SSE UI.
Phase 3 — Reservation	Seat model, Redis TTL, Redisson locks, reservation APIs.
Phase 4 — Collaboration	WebFlux, STOMP/WebSockets, room lifecycle, Redis Pub/Sub.
Phase 5 — Checkout	Order/Ledger, idempotency, Saga orchestration, Transactional Outbox.
Phase 6 — Events & Analytics	Kafka/MSK, Analytics Service, ClickHouse, audit dashboards.
Phase 7 — AWS productionization	EKS, ECR, ALB, WAF, CloudFront, S3, ElastiCache, MSK, IAM/KMS/Secrets Manager.
Phase 8 — Reliability	Autoscaling, observability, backups, DR, chaos tests and performance benchmarks.

17. Final Enterprise Architecture Narrative

EventPulse is an enterprise-grade distributed platform in which the user journey is the organizing principle. A user starts with an AI-assisted discovery experience, moves into an interactive seat-selection experience, optionally collaborates with friends in real time, receives a server-authoritative temporary reservation, and completes an idempotent financial workflow.

Behind that experience, the system deliberately separates responsibilities. The gateway handles ingress security and throttling. AI Discovery handles semantic retrieval and streamed responses. Collaboration handles transient real-time state. Reservation owns inventory concurrency. Order & Ledger owns financial workflow and compensation. Kafka/MSK carries durable domain events, while Analytics & Audit converts those events into analytical/audit data.

AWS provides the production foundation: global frontend delivery through CloudFront/S3, edge protection through WAF, ingress through ALB, container orchestration through EKS, managed Kafka through MSK, distributed Redis through ElastiCache, transactional PostgreSQL through Aurora/RDS, security through IAM/KMS/Secrets Manager, and operational visibility through CloudWatch.

The result is more than a collection of technologies. It is a coherent system where each technology has a defined responsibility, each user action has a backend owner, each critical failure has a recovery path, and each performance target has a measurable test strategy.

Recommended project pitch: “Designed and implemented an enterprise-grade AWS event-driven ticketing platform using Java 21, Spring Boot, EKS, MSK, Redis, PostgreSQL, Qdrant and ClickHouse, with AI-powered discovery, real-time collaborative seating, distributed seat locking, idempotent payments, Saga-based recovery and Transactional Outbox-driven event consistency.”

END OF EVENTPULSE ENTERPRISE FUNCTIONAL & TECHNICAL SPECIFICATION • VERSION 2.0